UNIVERSIDAD DE LOS LLANOS — INGENIERÍA DE SISTEMAS

Procesos Estocásticos (Código 603504) | Proyecto Final: Reconocedor de Caracteres HMM 2D/3D

FASE 2: DOCUMENTACIÓN MATEMÁTICA COMPLETA Y TRAZA DE EJECUCIÓN

Matemática base, matemática auxiliar, ejemplos reales de 1 iteración sobre el dataset y flujo end-to-end

Procedencia de los datos numéricos: Todos los ejemplos prácticos de este documento provienen de ejecuciones reales sobre el repositorio usando la muestra `a[0]` (`T = 75` símbolos) e `i[0]` de `data/glifo-dataset-2026-11-12.json` y los modelos entrenados `models/2d_a.json` y `models/2d_o.json`.

BLOQUE 1: Matemática Base Aplicada

1.1 Modelo Oculto de Markov Discreto λ_c y Topología Bakis

Para cada una de las 27 letras minúsculas del español `C = {a, ..., z}` se construye un HMM discreto independiente de primer orden en 2D y otro en 3D. Cada modelo posee $N_c \in [4, 11]$ estados ocultos secuenciales y un alfabeto de `|V| = M = 20` símbolos (`K = 16` direcciones angulares y 4 saltos con pluma levantada). Con topología **Bakis izquierda-derecha** (`init_bakis_hmm`, lín. 449–467), el trazo comienza siempre en S_0, solo permanece en S_i o avanza hacia S_{i+1}, S_{i+2}, y finaliza absorbido en S_{N-1}.

$$\lambda_c = (A_c, B_c, \pi_c), \qquad \pi_0 = 1,\ \pi_i = 0\ (i>0), \qquad a_{ij} = 0\ \forall j \notin \{i, i+1, i+2\}$$

$$\sum_{j=0}^{N-1} a_{ij} = 1, \qquad \sum_{k=0}^{M-1} b_j(k) = 1, \qquad a_{N-1,\,N-1} = 1$$

Ejemplo real de parámetros (fila i=0 de `models/2d_a.json`, N=8, M=20):

π = [1, 0, 0, 0, 0, 0, 0, 0]^T; $A_{0,:}$ = [0.871658, 0.117696, 0.010647, 0, 0, 0, 0, 0]; $B_{0,\,0..7}$ = [10^{-5}, 10^{-5}, 0.000117, 0.004204, 0.029828, 0.056934, 0.147441, 0.297486].

1.2 Evaluación: Algoritmo Forward con Escalado de Rabiner (`HMM.forward`, lín. 157–177)

Evaluar por fuerza bruta `P(O | λ)` requiere `O(N^T)` caminos. La programación dinámica Forward lo reduce a `O(N²T)` definiendo `α_t(i) = P(O_0..t, q_t=S_i | λ)`. Dado que multiplicar `T ≈ 70` probabilidades menores que 1 produce *underflow* en IEEE-754, se normaliza cada paso con el factor c_t. Por inducción matemática, la suma final unitaria de $\hat{\alpha}_{T-1}$ demuestra exactamente la identidad logarítmica:

$$\tilde{\alpha}_0(i) = \pi_i b_i(O_0), \qquad c_0 = \frac{1}{\sum_{i=0}^{N-1} \tilde{\alpha}_0(i)}, \qquad \hat{\alpha}_0(i) = c_0 \tilde{\alpha}_0(i)$$

$$\tilde{\alpha}_{t+1}(j) = \left[\sum_{i=0}^{N-1} \hat{\alpha}_t(i)\, a_{ij}\right] b_j(O_{t+1}), \qquad c_{t+1} = \frac{1}{\sum_{j=0}^{N-1} \tilde{\alpha}_{t+1}(j)}, \qquad \hat{\alpha}_{t+1}(j) = c_{t+1} \tilde{\alpha}_{t+1}(j)$$

$$\hat{\alpha}_t(i) = \left(\prod_{\tau=0}^{t} c_\tau\right)\alpha_t(i) \quad \Rightarrow \quad \log P(O \mid \lambda) = -\sum_{t=0}^{T-1}\log(c_t)$$

Ejemplo práctico de 1 iteración (t=0 → t=1 sobre `a[0]`, O_0=4, O_1=6 en `models/2d_a.json`):

1) **t=0:** $\tilde{\alpha}_0(0) = \pi_0 \cdot b_0(4) = 1.0 \times 0.029828 = 0.029828 \Rightarrow c_0 = 1/0.029828 =$ **33.525945** $\Rightarrow \hat{\alpha}_0 = [1.0, 0, ..., 0]^T$.
2) **Iteración t=0 → 1 (O_1=6):** con $b_0(6)=0.147441$ y $b_1(6)=b_2(6)=10^{-5}$:
• $\tilde{\alpha}_1(0) = (1.0 \times 0.871658) \times 0.147441 = 0.128518$; $\tilde{\alpha}_1(1) = (1.0 \times 0.117696) \times 10^{-5} = 1.177 \times 10^{-6}$; $\tilde{\alpha}_1(2) = (1.0 \times 0.010647) \times 10^{-5} = 1.065 \times 10^{-7}$.
• Suma = 0.128519 $\Rightarrow c_1 = 1/0.128519 =$ **7.780927** $\Rightarrow \hat{\alpha}_1 = c_1\tilde{\alpha}_1 =$ **$[0.999990, 9\times10^{-6}, 1\times10^{-6}, 0, ...]^T$**.
• Acumulado: $-\log(33.525945) - \log(7.780927) =$ **−5.563995** (al terminar T=75 pasos vale **−96.2750**).

1.3 Evaluación Retrospectiva: Backward con Escalado (`HMM.backward`, lín. 182–196)

Calcula las probabilidades futuras `β_t(i) = P(O_{t+1..T-1} | q_t=S_i, λ)` escaladas en sentido inverso con los mismos coeficientes `c_t` obtenidos en Forward, manteniendo escala estable y permitiendo su cancelación algebraica en Baum-Welch.

$$\hat{\beta}_{T-1}(i) = c_{T-1}, \qquad \hat{\beta}_t(i) = c_t \sum_{j=0}^{N-1} a_{ij}\, b_j(O_{t+1})\, \hat{\beta}_{t+1}(j) \quad (t = T-2, \ \dots, 0)$$

Ejemplo práctico de 1 iteración (t=73 desde T−1=74 en `a[0]`):

Entradas: O_{74}=0, c_{74}=12.325531, c_{73}=10.215491, $\hat{\beta}_{74}=[12.325531, ...]^T$, $b_6(0)$=0.000055, $b_7(0)$=0.081135.
• Estado absorbente i=7 (a_{77}=1.0): $\hat{\beta}_{73}(7)$ = 10.215491 × (1.0 × 0.081135 × 12.325531) = **10.215873**.
• Estado i=6 ($A_{6,6:8}$=[0.903331, 0.096669]): $\hat{\beta}_{73}(6)$ = 10.215491 × (0.903331×0.000055 + 0.096669×0.081135) × 12.325531 = **0.993772**.

1.4 Decodificación Óptima: Viterbi en Log-Espacio (`HMM.viterbi`, lín. 201–225)

Resuelve el problema de hallar la secuencia única de estados ocultos más probable `Q*` convirtiendo los productos en sumas logarítmicas y guardando los predecesores óptimos en la tabla `ψ_t(j)` para reconstruir el camino por backtracking.

$$\delta_0(i) = \log \pi_i + \log b_i(O_0), \qquad \delta_t(j) = \max_{0 \le i < N} [\delta_{t-1}(i) + \log a_{ij}] + \log b_j(O_t)$$

$$\psi_t(j) = \arg\max_{0 \le i < N} [\delta_{t-1}(i) + \log a_{ij}], \qquad q^*_{T-1} = \arg\max_i \delta_{T-1}(i), \qquad q_t^* = \psi_{t+1}(q^*_{t+1})$$

Ejemplo práctico de 1 iteración (t=1, O_0=4, O_1=6 en `models/2d_a.json`):

1) En t=0: $\delta_0(0) = \log(1) + \log(0.029828) =$ **−3.5123**; $\delta_0(1) = -702.3178$; $\delta_0(2) = -702.3091$.
2) Sumas candidatas $\delta_0(i) + \log a_{ij}$ para j ∈ {0,1,2}: desde i=0 dan [**−3.6497, −5.6520, −8.0548**]; desde i=1 dan [−1393.09, −702.45, −704.46] ⇒ punteros $\psi_1(0..2)$ = **[0, 0, 0]**.
3) Sumando $\log b_j(O_1=6)$ = [−1.9143, −11.5415, −11.5336]: $\delta_1(0..2)$ = **[−5.5640, −17.1935, −19.5884]** (en t=74 termina en q_{74}^*=7 con logP = **−98.463**).

1.5 Aprendizaje Offline: Baum-Welch / EM Multisecuencia (`HMM.fit`, lín. 230–334)

Maximiza iterativamente la verosimilitud conjunta sobre las `R` muestras de entrenamiento de cada letra. En el **Paso E** los factores de escala c_t se cancelan exactamente al calcular las ocupaciones posteriores $\gamma_t(i)$ y transiciones esperadas $\xi_t(i,j)$. En el **Paso M** se reestiman `A` (dentro de la máscara Bakis) y `B` manteniendo fijo $\pi = [1, 0, \ldots, 0]^\top$.

$$\gamma_t(i) = \frac{\hat{\alpha}_t(i)\hat{\beta}_t(i)}{c_t}, \qquad \xi_t(i,j) = \frac{\hat{\alpha}_t(i)\, a_{ij}\, b_j(O_{t+1})\, \hat{\beta}_{t+1}(j)}{\sum_{p,q} \hat{\alpha}_t(p)\, a_{pq}\, b_q(O_{t+1})\, \hat{\beta}_{t+1}(q)}$$

$$\bar{a}_{ij} = \frac{\sum_{r=1}^{R}\sum_{t=0}^{T_r-2} \xi_t^{(r)}(i,j)}{\sum_{r=1}^{R}\sum_{t=0}^{T_r-2} \gamma_t^{(r)}(i)}, \qquad \bar{b}_j(k) = \frac{\sum_{r=1}^{R}\sum_{t:\,O_t^{(r)}=k} \gamma_t^{(r)}(j) + \epsilon}{\sum_{r=1}^{R}\sum_{t=0}^{T_r-1} \gamma_t^{(r)}(j) + M\epsilon}$$

Ejemplo práctico de 1 iteración EM completa (época 1 sobre `a[0]`, fila i=0 desde `init_segmental_bakis_hmm`):

Valores iniciales: $A_{0,0..2}$ = [0.883495, 0.106796, 0.009709], $B_{0,4}$ = 0.091667.
• **Paso E:** en t=0, $\gamma_0(0)$=1.0 y $\xi_0(0,0)$=1.0. Acumulando los 75 pasos: $A_{den}[0]$ = 9.293533, $A_{num}[0,0..2]$ = [8.293533, 0.999992, 0.000008], $B_{den}[0]$ = 9.293533, $B_{num}[0,4]$ = 1.0.
• **Paso M** (`pseudo_count=0.01`, M=20): tras dividir y renormalizar en las 3 celdas Bakis, $A_{0,0..2}$ actualiza a **[0.876115, 0.114176, 0.009710]** y $B_{0,4}$ = (1.0 + 0.01) / (9.293533 + 20·0.01) = **0.106388**.

1.6 Clasificación Bayesiana Normalizada por Longitud y Proyección Geométrica

Con equiprobabilidad a priori `P(c)=1/27` sobre el alfabeto cerrado `C`, el clasificador calcula siempre la distribución estadística posterior completa `P(c | O₁:t, g)` (garantizando `Σ_{c∈C} P(c|...) = 100%` sobre cualquier trazo dibujado). Como `Score(c)` es una magnitud **extensiva en `T`**, un Softmax sin escala satura a `100.0%` en cuanto $S_1 - S_2 > 7$. Para evitar la saturación sin alterar el `argmax` ni rechazar trazos, las probabilidades se escalan por longitud y ajuste global: mientras se dibuja (`complete=False`) se usa $\tau_{vivo} = \max(1, T/18)$; al finalizar (`complete=True`) se usa $\tau_{final} = \max(1, 0.25 \cdot T) \cdot \max(1, (-S^*/T)/1.8)$, donde `1.8 nats/símbolo` corresponde al percentil 95 de ajuste en letras válidas. Así, en letras bien trazadas ($-S^*/T \le 1.8$) el factor es `1.0`, mientras que en garabatos extremos la incertidumbre se reparte de manera realista. Para proyectar desde q_t^*, la permanencia en S_j sigue una **Geométrica** de media $E[d_j]=1/(1-a_{jj})$.

$$P(c \mid O_{1:t}, g) = \frac{\exp(\text{Score}(c)/\tau(T))}{\sum_{c' \in C}\exp(\text{Score}(c')/\tau(T))} = \frac{\left[P(O_{1:t} \mid \lambda_c)e^{-P_{tot}(c)}\right]^{1/\tau(T)}}{\sum_{c'}\left[P(O_{1:t} \mid \lambda_{c'})e^{-P_{tot}(c')}\right]^{1/\tau(T)}}$$

$$\tau_{\text{vivo}} = \max\left(1.0,\ \tfrac{T}{18.0}\right), \qquad \tau_{\text{final}} = \max(1.0,\ 0.25\,T) \cdot \max\left(1.0,\ \tfrac{-\text{Score}(c^*)/T}{1.8}\right)$$

$$E[d_j] = \tfrac{1}{1-a_{jj}}, \qquad k_j^* = \arg\max_k b_j(k), \qquad d_{\text{rem}}(q_t^*) = \max\left(0,\ \text{round}(E[d_{q_t^*}]) - \text{spent}\right)$$

Ejemplo práctico de 1 ciclo en vivo + proyección (prefijo 50% de `a[0]`, T=36, τ_{vivo}=36/18=2.0):

En T=36 (`complete=False`), el ganador parcial es `'c'` (Score=−51.11) frente a `'a'` (Score=−51.17). Con τ_{vivo}=2.0, el softmax normalizado por longitud arroja **P(c)=50.27%**, **P(a)=48.78%**, **P(d)=0.49%** y **P(e)=0.22%** (reflejando la ambigüedad real de un trazo abierto), con Viterbi en `state=3` y `spent=6`.
1 iteración de proyección en `state=3`: a_{33}=0.878421 ⇒ E[d_3]=1/(1−0.878421)=8.2251 → round=8. Restando `spent=6` quedan **2** emisiones del símbolo modal argmax_k $B_{3,k}$ = **15** (`[15, 15]`) y avanza a `state=4`.

1.7 Geometría Diferencial sobre la Esfera S^2 (`FeatureExtractor3D`, lín. 331–537)

Sobre la esfera unitaria los puntos en marco ecuatorial se parametrizan por colatitud `θ ∈ [0, π]` y azimut `ϕ`, con métrica `ds² = dθ² + sin²θ dϕ²`. El remuestreo sigue círculos máximos mediante interpolación esférica **SLERP**; los rumbos se miden en la base tangente ortonormal `{ê_ϕ, −ê_θ}`; y la reconstrucción futura avanza sobre S^2 mediante el **mapa exponencial**:

$$\Omega = \arccos(\hat{p}_0 \cdot \hat{p}_1), \qquad \hat{q}(t) = \frac{\sin((1-t)\Omega)}{\sin\Omega}\hat{p}_0 + \frac{\sin(t\Omega)}{\sin\Omega}\hat{p}_1$$

$$\psi_m = \operatorname{atan2}(-d\theta_m,\ d\phi_m \cdot \sin\bar{\theta}_m), \qquad \hat{p}_{\text{next}} = \cos(\delta)\hat{p} + \sin(\delta)\hat{t}(\psi)$$

Ejemplo práctico de 1 paso geodésico (`a[0]` proyectada en S^2 con cámara (0°, 18°)):

• **SLERP (t=0.5)** entre $\hat{p}_0$=[0.855161, 0.329472, 0.400184] y $\hat{p}_{25}$=[0.796143, 0.157716, 0.584194]: Ω = arccos($\hat{p}_0 \cdot \hat{p}_{25}$) = 0.259265 rad, pesos w_0=w_1=sin(0.129632)/sin(0.259265)=0.504231 ⇒ $\hat{q}$(0.5) = **[0.832638, 0.245655, 0.496353]**.
• **Rumbo tangente** entre (1.159079, 0.367748) y (1.082304, 0.356779): dθ=−0.076774, dφ=−0.010969, θ̄=1.120692 ⇒ ψ = atan2(+0.076774, −0.010969·sin(1.120692)) = 1.698739 rad (97.33° → **símbolo 4**).
• **Mapa exponencial** desde [0.881204, 0.468079, −0.066197] con δ=0.078505 rad y símbolo 4 avanza exactamente a **[0.883074, 0.469073, 0.012259]** (norma = 1.0000).

BLOQUE 2: Matemática Auxiliar / Extra y Ajustes Numéricos

2.1 Normalización Dual con Inversión Y, Piso de Escala y Suavizado en Bordes Fijos

Las pantallas miden `+Y` hacia abajo; invertir `y ← −y` alinea los ejes con `atan2`. Durante el trazo activo (`realtime=True`) se impone un piso de escala (`250.0 px` en 2D; `0.52 rad` ≈ 30° en 3D, acorde al recuadro guía naranja de ±30° y paso geodésico mínimo `0.005 rad` en proyección) para no comprimir un segmento inicial en `[0,1]²`; al soltar (`span ≥ 35 px` o `≥ 12 pts`) el piso cae a `10⁻⁶` logrando invarianza total al tamaño. Luego, un promedio móvil simétrico *O(n)* basado en `np.cumsum` filtra temblores anclando los bordes.

$$p' = \frac{(p_x,\ -p_y) - p_{\min}}{\max(w, h, \text{piso})}, \qquad \tilde{p}_i = \frac{1}{2r_i+1}\sum_{j=-r_i}^{r_i} p'_{i+j}, \quad r_i = \min\left(\lfloor w/2 \rfloor,\ i,\ n-1-i\right)$$

Ejemplo práctico en `a[0]` (primer punto crudo [557.7, 486.0], p_{min}=[382.7, −680.6], scale=255.3):

• Normalizado: x'$_0$ = (557.7 − 382.7)/255.3 = **0.685468**; y'$_0$ = (−486.0 − (−680.6))/255.3 = **0.762241**.

• Suavizado: en el borde i=0, r_0=min(1,0,376)=0 ⇒ $\tilde{p}_0$ = **[0.685468, 0.762241]** (anclado exacto); en i=3, r_3=1 ⇒ $\tilde{p}_3$ = (p'_2+p'_3+p'_4)/3 = **[0.685468, 0.783262]**.

2.2 Remuestreo por Longitud de Arco, Cuantización Centrada y Filtro de Micro-Saltos

Parametrizar por arco acumulado s_i con paso constante `D = 0.05` elimina la dependencia de la velocidad del usuario (el timestamp `t` nunca entra al modelo). Al cuantizar ángulos, se suma medio bin (`+π/K` o `+π/4`) para centrar los sectores en los ejes cardinales y evitar rebotes sobre 0° o 90°. Micro-saltos entre sub-trazos menores que `D` son descartados.

$$s_i = \sum_{k=1}^{i} \|\tilde{p}_k - \tilde{p}_{k-1}\|_2, \qquad u_m = m \cdot D, \qquad q_m = \tilde{p}_k + \tfrac{u_m - s_k}{s_{k+1} - s_k}(\tilde{p}_{k+1} - \tilde{p}_k)$$

$$o_{\text{dir}}(\phi) = \left\lfloor \tfrac{(\phi + \pi/K) \bmod 2\pi}{2\pi/K} \right\rfloor \bmod K, \qquad o_{\text{jump}}(\phi) = K + \left(\left\lfloor \tfrac{(\phi + \pi/4) \bmod 2\pi}{\pi/2} \right\rfloor \bmod 4 \right)$$

Ejemplo práctico de 1 interpolación (u_1=0.05) y 1 cuantización angular + salto:

• **Remuestreo u_1=0.05:** cae en segmento k=5 (s_5=0.039855, s_6=0.050365); fracción (0.05−0.039855)/0.010510 = **0.965284** ⇒ q_1 = [0.683118, 0.801802] + 0.965284([−0.004700, 0.009401]) = **[0.678581, 0.810876]**.
• **Cuantización q_0→q_1:** Δ=[−0.006887, 0.048636], φ=1.711472 rad (98.06°) ⇒ ⌊(1.711472+0.196350)/0.392699⌋ = **4** (↑).
• **Salto cuerpo→punto en `i[0]`:** Δ=[0.036846, 1.0] (dist=1.0007 ≥ 0.05, 87.89°) ⇒ 16 + 1 = **17** (J↑).

2.3 Descriptor Morfológico 7D y Penalizaciones Auxiliares al Score (con P_{dur} Huberizada)

Para desempatar pares con cadenas direccionales similares (`a` vs `o`, `v` vs `u`, `l` vs `i`), se integra al log-score una penalización de duración Huberizada $P_{dur} = h_4(z)$ (cuadrática en `|z|≤4` y lineal C^1 en `|z|>4` para impedir que un crecimiento $O(T^2)$ domine a la temperatura $O(T)$ en garabatos muy extensos; en parciales solo actúa si `T > μ+1.5σ`), una penalización sobre el vector global 7D P_{shape} (peso `1.4` completo o rampa `≤0.45` desde `0.85μ` en vivo) y una penalización estructural por saltos P_{jump} (`+16/salto` en modelos monotrazo y `+24` si faltan sub-trazos al finalizar un modelo multitrazo).

$$\text{Score}(c) = \log P(O_{1:t} \mid \lambda_c) - P_{\text{dur}}(T; c) - P_{\text{shape}}(g; c) - P_{\text{jump}}(O_{1:t}; c)$$

$$g = \left[\log\left(\text{clip}(\tfrac{w}{h}, 0.08, 12.0)\right),\ x'_{\text{ini}},\ y'_{\text{ini}},\ x'_{\text{fin}},\ y'_{\text{fin}},\ \bar{x}',\ \bar{y}' \right]^T$$

$$P_{\text{dur}}(T) = h_4\left(\tfrac{T - \mu_c}{\max(\sigma_c, 4.0)}\right), \quad h_4(z) = \tfrac{z^2}{2}\ (|z| \le 4)\ ;\ 4|z| - 8\ (|z| > 4), \quad P_{\text{shape}}(g) = \tfrac{w}{2} \sum_{d=1}^{7} \left(\tfrac{g_d - \mu_{c,d}}{\max(\sigma_{c,d}, 10^{-6})}\right)^2$$

Ejemplo práctico comparando `a[0]` completo (T=75, 1 salto) contra modelos `a` y `o`:

Vector real: g = [−0.10262, 0.75955, 0.76224, 1.0, 0.00666, 0.49073, 0.46050].
• **Modelo `a`** (μ=68.957, σ=4.878, z=1.239≤4): P_{dur} = **0.7673**, P_{shape} = **1.3086**, P_{jump} = **0.0** ⇒ Score(a) = −96.2750 − 0.7673 − 1.3086 = **−98.3510**.
• **Modelo `o`** (μ=67.387, σ=7.510, z=1.014≤4): como `o` cierra arriba ($\mu_{shape,4}$=0.9327) y no tiene saltos, P_{dur}=0.5138,

P_{shape}=**36.1211** y P_{jump}=**16.0** ⇒ Score(o) = −216.726 − 52.635 = **−269.3609**.

2.4 Log-Sum-Exp Normalizado por Longitud y Ajuste Global, Guardas `1e-300` e Inicialización

Para evitar *underflow* al exponenciar scores negativos y eliminar la saturación artificial hacia 100.0% tanto en letras ambiguas como en trazos caóticos muy largos, `HMMClassifier.classify` aplica el truco Log-Sum-Exp con temperatura τ_{vivo}=`max(1, T/18)` en vivo y τ_{final}=`max(1, 0.25·T)·max(1, (-S*/T)/1.8)` al completar, conservando intacto el `argmax` de las 27 clases y garantizando siempre Σ_c `P(c|O) = 100%`. Todo logaritmo y divisor se protege con `1e-300`. En entrenamiento, `init_segmental_bakis_hmm` divide cada muestra en `N` bloques uniformes y `HMM.fit` suaviza circularmente los `K` sectores de `B[:, :K]` (pesos `0.11/0.78/0.11`) sin alterar los 4 saltos.

$$P(c \mid O) = \exp\left(\frac{\text{Score}(c)}{\tau} - m - \log\left[\sum_{c' \in \mathcal{C}} e^{\text{Score}(c')/\tau - m}\right]\right), \qquad m = \max_{c' \in \mathcal{C}} \frac{\text{Score}(c')}{\tau}$$

$$\tau_{\text{final}} = \max(1.0,\ 0.25\,T) \cdot \max\left(1.0,\ \tfrac{-S^*/T}{1.8}\right), \qquad b_j^{\text{suav}}(k) = 0.78\, b_j(k) + 0.11\, b_j((k \pm 1) \bmod K)$$

Ejemplos prácticos reales: Log-Sum-Exp Calibrado (`a[0]` vs garabato largo denso) y Suavizado Circular:

• **Letra válida `a[0]` completa (T=75):** `S*=S`$_a$`=-98.351` (−S*/T=1.311≤1.8 ⇒ factor=1.0, τ_{final}=18.75) frente a S_q=`-132.224` y S_d=`-154.562`. En vez de saturar al 100.00% arroja una distribución realista: **P(a)=81.82%, P(q)=13.44%, P(d)=4.08%, P(g)=0.60%**.
• **Garabato denso extenso (T=755):** `S*=-2948.841` (−S*/T=3.906 > 1.8 ⇒ factor=2.170, τ_{final}=409.56). Gracias a la Huberización h_4 (sin explosiones $O(T^2)$) y al escalado por ajuste global, no satura al 100% y reparte la estadística: **P(i)=43.16%, P(d)=21.18%, P(b)=12.20%, P(k)=10.18%** (suma `100.00%`).
• **Suavizado circular k=6 (fila 0):** $B_{0,5..7}$=[0.044681, 0.236170, 0.321277] ⇒ $B^{suav}_{0,6}$ = 0.11(0.044681)+0.78(0.236170)+0.11(0.321277) = **0.224468**.

2.5 Marco $R_{cam} \in SO(3)$, `np.unwrap`, Ray-Casting Exacto y Aumento Afín

En 3D, `build_camera_matrix` rota la vista actual `(az, el)` al ecuador frontal evitando la singularidad de los polos; `np.unwrap` corrige saltos ±2π en φ; `resample_geodesic` conmuta a LERP si `sin Ω ≤ 10`$^{-8}$; y los clics en pantalla se transforman con `(P·V)`$^{-1}$ resolviendo la cuadrática rayo-esfera. En entrenamiento se aumentan muestras con matriz afín `M = R(θ)·Sh(k)·S(`s_x`, `s_y`)` y, al completar trazos en GUI, si q_t`* ≥ N-1` y `T ≥ 0.75`μ_c se acota la proyección a `future_symbols[:1]`.

$$\|o + t\,d\|^2 = R^2 \ \Rightarrow\ at^2 + bt + c = 0 \quad (a = d \cdot d,\ b = 2(o \cdot d),\ c = o \cdot o - R^2)$$

$$M = R(\theta)\,\text{Sh}(k)\,S(s_x, s_y), \qquad p_{\text{aug}} = (p - c_{\text{box}})M^T + c_{\text{box}} + \mathcal{N}(0, (\sigma \cdot \text{span})^2)$$

Ejemplos prácticos de 1 cálculo real (3D y aumento afín): • **Rotación cámara (az=0°, el=18°):** R_{cam}[0,:]=[0.951057, 0, 0.309017] lleva [0.689643, 0.329472, 0.644857] a **[0.855161, 0.329472, 0.400184]**.
• **Ray-casting (R=1):** o=[2.05, 0, 0.2], d=[−1.0, 0.1, −0.05] ⇒ a=1.0125, b=−4.12, c=3.2425 ⇒ impacto frontal **[0.983418, 0.106658, 0.146671]**.
• **Aumento afín (seed=100 en `a[0]`):** θ=4.69°, s=(1.0193, 0.9578), k=−0.0731 ⇒ [557.7, 486.0] pasa a **[568.131, 497.290]**.

BLOQUE 3: Traza de Ejecución End-to-End

1. **Entrenamiento Offline (`python -m training.train --mode all`):** Carga `config.toml` y 2,560 muestras de `glifo-dataset-2026-11-12.json`; en 3D proyecta al ecuador con apertura `0.52 rad` (≈30°); aumenta clases con <80 muestras; extrae secuencias y descriptores 7D; inicializa segmentariamente cada uno de los 54 modelos, ejecuta Baum-Welch (hasta 30 épocas) + suavizado circular y guarda `models/2d_*.json` y `3d_*.json`.
2. **Inferencia en Tiempo Real (`main.py` → `View2D` / `View3D`):** Al pulsar y arrastrar el ratón ($\Delta^2 \geq 2.25$ px en 2D o $d \geq 0.012$ sobre S^2 por ray-casting en 3D), `_evaluate_realtime()` extrae símbolos con piso de escala (`realtime=True`), ejecuta `predict_and_project(..., complete=False)` sobre los 27 modelos, actualiza `RankingPanel` y pinta la proyección punteada verde. Al soltar el clic (`is_complete=True`), desbloquea la invarianza de escala (piso 10^{-6}), activa peso completo de morfología (`w=1.4`) y saltos, y recorta la cola proyectada si ya alcanzó el estado final.